南京理工大学钱学森学院鼎新班

程序设计基础(I)课程设计 报告

班　　级 924117A001

学生姓名 薄闻乐

学　　号 924104780512

学生姓名 魏舟

学　　号 924101760633

学生姓名 尹杨阳

学　　号 924117A00115

研究方向 装备系统动力学

院士团队 芮筱亭院士团队

起止时间 2025.4.1-2025.5.23

指导教师 衷 宜

2025 年 **5** 月

目 录

1 课题：名称 1
1.1 课题功能及来源 1
1.2 开发平台 1
1.3 总体方案 1
1.3.1 类图 2
1.3.2 总体流程图 2
程序总体思路，总的流程图或框图 ..... **错误!未定义书签。**
1.4 详细设计 3
1.5 测试与运行 4
1.6 调试记录 30
1.7 版本记录（若有） 32
2 分工情况 32
3 心得体会 32

1 课题：矩阵运算规则下的棋盘棋子博弈模型构建

1.1 课题功能及来源

课题所解决的问题

该课题主要围绕矩阵运算和博弈机制，设计并实现了一个矩阵对战游戏，解决了如何将矩阵的数学运算与游戏规则相结合，以实现具有趣味性和策略性的游戏体验的问题。

主要功能

1.矩阵初始化：创建矩阵类 Matrix，能够随机生成矩阵元素，也可通过输入进行初始化。
2.矩阵运算：支持矩阵的转置、求行列式、求秩、求逆等基本运算，以及矩阵的加法、减法和乘法运算。
3.游戏界面绘制：使用 Windows GDI 绘制游戏界面，包括棋盘、棋子、按钮等元素，展示游戏状态和当前玩家信息。
4.棋子操作：玩家可以选择棋子，进行移动和使用技能的操作。移动棋子时，根据目标位置是否有其他棋子，进行合并或对战操作；使用技能时，可对选中的棋子应用不同的矩阵运算技能。
5.游戏规则判定：根据矩阵的行列式值判断对战结果，当一方剩余棋子数量小于 3 时，比较特定棋子的行列式值决定游戏胜负。
6.信息展示：可以查看选中棋子的矩阵信息，包括矩阵元素、行列式值和秩。
7.游戏结束处理：当满足游戏结束条件时，显示游戏结果，包括获胜方和双方剩余棋子数量。

课题来源

结合我们组去年开发的高维矩阵乘法计算器，和类似与军棋的玩法，自创了一款游戏。未来可能会将矩阵向高维拓展，增加游戏随机性的同时，可以从用户的走法中得到更为简便的高维矩阵运算形式。

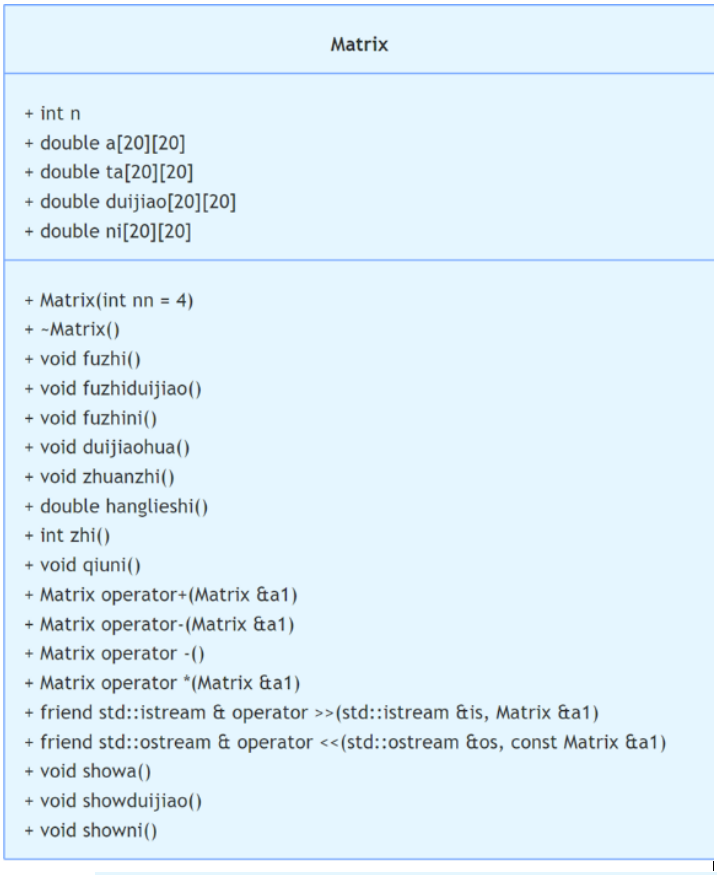
借鉴的部分

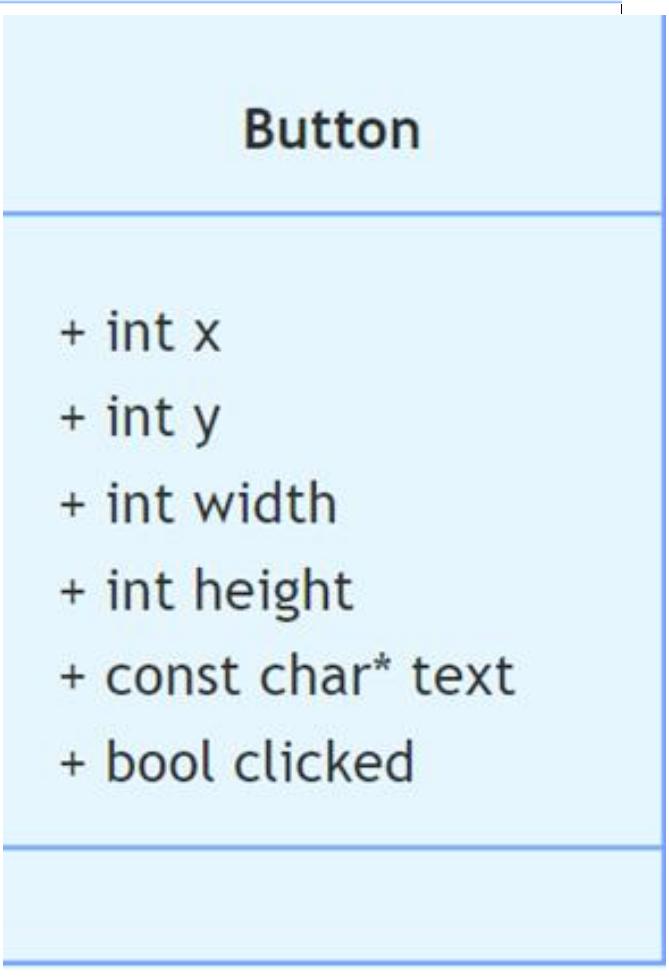
可视化部分借鉴 B 站视频：Windows GDI 编程教程、Windows 图形界面编程等；
可视化代码部分调试阶段使用 AI（豆包）

1.2 开发平台

Win11，Dev-C++ 5.11　TDM-GCC 4.9.2 64-bit Release1.3

1.3.1 类图

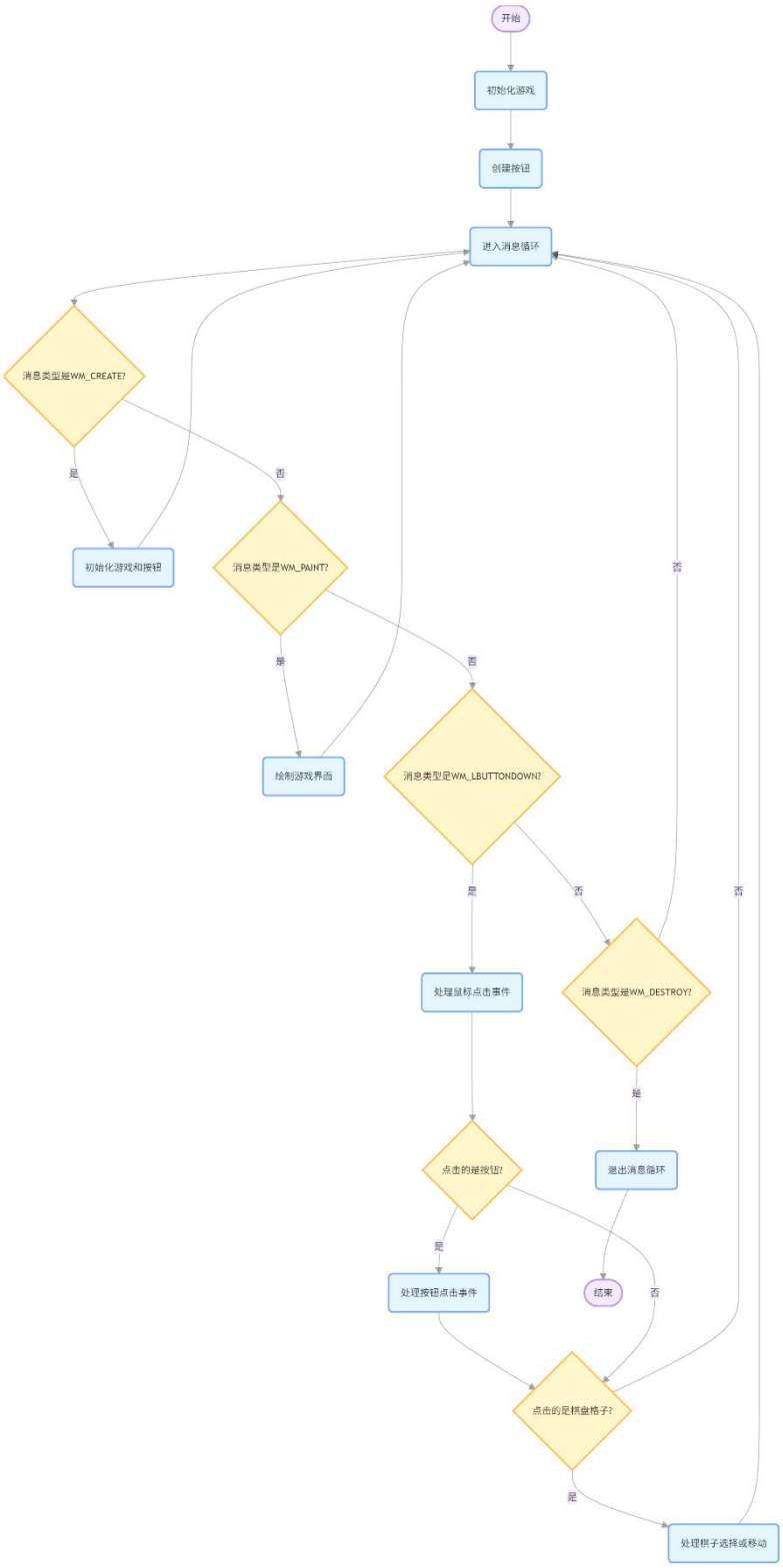

Matrix
+ int n
+ double a[20][20]
+ double ta[20][20]
+ double duijiao[20][20]
+ double ni[20][20]
+ Matrix(int nn = 4)
+ ~Matrix()
+ void fuzhi()
+ void fuzhiduijiao()
+ void fuzhini()
+ void duijiaohua()
+ void zhuanzhi()
+ double hanglieshi()
+ int zhi()
+ void qiuni()
+ Matrix operator+(Matrix &a1)
+ Matrix operator-(Matrix &a1)
+ Matrix operator -()
+ Matrix operator *(Matrix &a1)
+ friend std::istream & operator >>(std::istream &is, Matrix &a1)
+ friend std::ostream & operator <<(std::ostream &os, const Matrix &a1)
+ void showa()
+ void showduijiao()
+ void showni()


Button
+ int x
+ int y
+ int width
+ int height
+ const char* text
+ bool clicked

1.3.2 总体流程图

Work() -> 初始化技能、棋盘与 20 个棋子 ->轮流操作（查看->技能->移动） 直到双方只剩下一枚棋子 ->结算

可视化流程图：



1.4 详细设计

```
void work()
{
    cout<<"=====  矩阵对战游戏  ====="<<endl;
    cout<<"游戏规则"<<endl;
    cout<<"1. 玩家 1 控制 1-10 号棋子，玩家 2 控制 11-20 号棋子"<<endl;
    cout<<"2. 棋子只能直线移动"<<endl;
    cout<<"3. 玩家可以使用技能增强棋子"<<endl;
    cout<<"4. 比较 3 号和 18 号棋子的行列式大小，小者失败"<<endl;
    cout<<"======================="<<endl;
    int amap[20][20],alivecnt,sx,sy,ex,ey,num,opt1,opt2,opt3;
    bool jineng[10][5] = {0};//1.加 E 2.求逆  3.转置  4.对角化  5.加负 E
    for (int i=1;i<=5;i++)
    {
        jineng[i][1]=1;
        jineng[i][2]=1;
    }

    // 初始化棋盘
    for (int i=1;i<=5;i++)
    {
        for (int j=1;j<=5;j++)
        {
            amap[i][j]=-1;
        }
    }

    // 放置玩家 1 的棋子
    for (int i=1;i<=2;i++)
    {
        for (int j=1;j<=5;j++)
        {
            amap[i][j]=(i-1)*5+j;
        }
    }

    // 放置玩家 2 的棋子
    for (int i=4;i<=5;i++)
    {
        for (int j=1;j<=5;j++)
        {
            amap[i][j]=(i-3)*5+j+5;
        }
```

```
}

// 创建单位矩阵 E
Matrix ee(4);
for (int i=1;i<=4;i++)
{
	for (int j=1;j<=4;j++)
	{
		if(i==j)
		{
			ee.a[i][j]=1.0;
		}
		else
		{
			ee.a[i][j]=0;
		}
	}
}

// 初始化随机数种子
srand(time(NULL));

// 生成 20 个随机矩阵
Matrix qizi[30];
for(int i = 1; i <= 20; i++) {
	qizi[i] = Matrix(4);
}

// 输出所有矩阵
cout << "初始棋子矩阵如下：" << endl;
for(int i = 1; i <= 20; i++) {
	cout << "第 " << i << " 个矩阵：" << endl;
	qizi[i].showa();
	cout << endl;
}

alivecnt=20;
cout<<"初始棋盘状态:"<<endl;
for(int i=1;i<=5;i++)
{
	for(int j=1;j<=5;j++)
	{
		if(amap[i][j]==-1)
		{
```

```
                cout<<"空 ";
            }
            else
            {
                cout<<amap[i][j]<<' ';
            }
        }
        cout<<endl;
    }
    while(1)
    {
        if(alivecnt<=2)
        {
            break;
        }
        while(1)
        {
            //玩家 1 操作
            cout<<"===== 玩家 1 回合 ====="<<endl;
            cout<<"请选择操作：1.查看棋子 2.使用技能 3.查看棋盘 -1.结束查看
"<<endl;
            while(!(cin>>opt1))//1.查看 qizi 2.查看技能 3.查看棋盘 -1.结束
            {
                cin.clear();
                cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                cout<<"输入无效，请输入一个有效的整数: ";
            }
            if(opt1==-1)
            {
                break;
            }
            if(opt1==3)
              {
                for (int i=1;i<=5;i++)
                {
                    for (int j=1;j<=5;j++)
                    {
                        if(amap[i][j]==-1)
                        {
                            cout<<"空 ";
                        }
                        else
                        {
                            cout<<amap[i][j]<<' ';
```

```
                    }
                }
                cout<<endl;
            }
        }
        if(opt1==1)
        {
            cout<<"请输入你要查看棋子的位置（行 列）："<<endl;
            while(!(cin>>sx>>sy))
            {
                cin.clear();
                cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                cout<<"输入无效，请输入两个有效的整数: ";
            }
            if(amap[sx][sy]==-1)
            {
                cout<<"该位置没有棋子！"<<endl;
                continue;
            }
            while(1)
            {
                cout<<"请选择查看选项：-1.返回 1.查看矩阵 2.查看行列式 3.查看逆矩阵 4.查看对角化"<<endl;//-1.结束 1.看 a 2.看行列式 3.看逆 4.看对角
                while(!(cin>>opt2))
                {
                    cin.clear();
                    cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                    cout<<"输入无效，请输入一个有效的整数: ";
                }
                if(opt2==-1)
                {
                    break;
                }
                if(opt2==1)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"的矩阵为"<<endl;
                    qizi[amap[sx][sy]].showa();
                }
                if(opt2==2)
                {
                    cout<<" 棋 子 "<<amap[sx][sy]<<" 的 行 列 式 为 ："<<qizi[amap[sx][sy]].hanglieshi()<<endl;
                }
                if(opt2==3)
```

```
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"的逆矩阵为"<<endl;
                        qizi[amap[sx][sy]].showni();
                    }
                    if(opt2==4)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"的对角化矩阵为"<<endl;
                        qizi[amap[sx][sy]].showduijiao();
                    }
                }
            }
            if(opt1==2)
            {
                cout<<"技能："<<endl;//-1 结束 1.加 E 2.逆 3.转置 4.对角 5.减 E
                cout<<"1.当前位置加 E 2.求逆 3.转置 4.对角化 5.当前位置加负
E"<<endl;
                while(1)
                {
                    cout<<"请选择技能(-1 退出)：";
                    while(!(cin>>opt2))
                    {
                        cin.clear();
                        cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                        cout<<"输入无效，请输入一个有效的整数: ";
                    }
                    if(opt2==-1)
                    {
                        break;
                    }
                    if(jineng[opt2][1]==0)
                    {
                        cout<<"该技能已使用！"<<endl;
                        continue;
                    }
                    cout<<"请输入你要使用技能的棋子位置（行 列）：";
                    while(!(cin>>sx>>sy))
                    {
                        cin.clear();
                        cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                        cout<<"输入无效，请输入两个有效的整数: ";
                    }
                    if(amap[sx][sy]<1 || amap[sx][sy]>10)
                    {
                        cout<<"你只能选择玩家 1 的棋子！"<<endl;
```

```
                    continue;
                }
                jineng[opt2][1]=0;
                if(opt2==1)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"使用加 E 技能！"<<endl;
                    qizi[amap[sx][sy]]=qizi[amap[sx][sy]]+ee;
                }
                if(opt2==2)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"使用求逆技能！"<<endl;
                    qizi[amap[sx][sy]].fuzhini();
                }
                if(opt2==3)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"使用转置技能！"<<endl;
                    qizi[amap[sx][sy]].zhuanzhi();
                }
                if(opt2==4)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"使用对角化技能！"<<endl;
                    qizi[amap[sx][sy]].fuzhiduijiao();
                }
                if(opt2==5)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"使用加负 E 技能！"<<endl;
                    ee=-ee;
                    qizi[amap[sx][sy]]=qizi[amap[sx][sy]]+ee;
                    ee=-ee;
                }
            }
        }
    }

    cout<<"请输入棋子移动信息（起始行 起始列 目标行 目标列）（不可移动 3
号）：";
    while(!(cin>>sx>>sy>>ex>>ey))//玩家 1 最后移动
    {
        cin.clear();
        cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
        cout<<"输入无效，请输入四个有效的整数: ";
    }
    while(1)
    {
```

```
            if(amap[sx][sy]<=10 && amap[sx][sy]!=3 && (sx==ex || sy==ey) && sx>0 && sx<=5 && sy>0 && sy<=5 && ex>0 && ex<=5 && ey>0 && ey<=5)
            {
                break;
            }
            cout<<"无效的移动，请重新输入：";
            while(!(cin>>sx>>sy>>ex>>ey))
            {
                cin.clear();
                cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                cout<<"输入无效，请输入四个有效的整数: ";
            }
        }
        num=amap[sx][sy];
        amap[sx][sy]=-1;
        while(1)
        {
            if(sx==ex && sy==ey)
            {
                break;
            }
            sx=sx+(ex-sx)/(abs(ex-sx));
            sy=sy+(ey-sy)/(abs(ey-sy));
            if(amap[sx][sy]!=-1)
            {
                break;
            }
        }
        if(amap[sx][sy]==-1)
        {
            amap[sx][sy]=num;
        }
        else
        {
            if(amap[sx][sy]<=10)
            {
                qizi[amap[sx][sy]]=qizi[amap[sx][sy]]+qizi[num];
            }
            else
            {
                qizi[amap[sx][sy]]=qizi[amap[sx][sy]]*qizi[num];
            }
            alivecnt--;
        }
```

```
        cout<<"移动完成！"<<endl;

        if(alivecnt<=2)
        {
            break;
        }
        while(1)
        {
            //玩家 2 操作
            cout<<"===== 玩家 2 回合 ====="<<endl;
            cout<<"请选择操作：1.查看棋子 2.使用技能 3.查看棋盘 -1.结束查看
"<<endl;
            while(!(cin>>opt1))//1.查看 qizi 2.查看技能 3.查看棋盘 -1.结束
            {
                cin.clear();
                cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                cout<<"输入无效，请输入一个有效的整数: ";
            }
            if(opt1==-1)
            {
                break;
            }
            if(opt1==3)
              {
                for (int i=1;i<=5;i++)
                {
                    for (int j=1;j<=5;j++)
                    {
                        if(amap[i][j]==-1)
                        {
                            cout<<"空 ";
                        }
                        else
                        {
                            cout<<amap[i][j]<<' ';
                        }
                    }
                    cout<<endl;
                }
            }
            if(opt1==1)
            {
                cout<<"请输入你要查看棋子的位置（行 列）："<<endl;
                while(!(cin>>sx>>sy))
```

```
                {
                    cin.clear();
                    cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                    cout<<"输入无效，请输入两个有效的整数: ";
                }
                if(amap[sx][sy]==-1)
                {
                    cout<<"该位置没有棋子！"<<endl;
                    continue;
                }
                while(1)
                {
                    cout<<"请选择查看选项：-1.返回 1.查看矩阵 2.查看行列式 3.查看逆矩阵 4.查看对角化"<<endl;//-1.结束 1.看 a 2.看行列式 3.看逆 4.看对角
                    while(!(cin>>opt2))
                    {
                        cin.clear();
                        cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                        cout<<"输入无效，请输入一个有效的整数: ";
                    }
                    if(opt2==-1)
                    {
                        break;
                    }
                    if(opt2==1)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"的矩阵为"<<endl;
                        qizi[amap[sx][sy]].showa();
                    }
                    if(opt2==2)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"的行列式为："<<qizi[amap[sx][sy]].hanglieshi()<<endl;
                    }
                    if(opt2==3)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"的逆矩阵为"<<endl;
                        qizi[amap[sx][sy]].showni();
                    }
                    if(opt2==4)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"的对角化矩阵为"<<endl;
                        qizi[amap[sx][sy]].showduijiao();
                    }
```

```
            }
        }
        if(opt1==2)
        {
            cout<<"技能："<<endl;//-1 结束 1.加 E 2.逆 3.转置 4.对角 5.减 E
            cout<<"1.当前位置加 E 2.求逆 3.转置 4.对角化 5.当前位置加负
E"<<endl;
            while(1)
            {
                cout<<"请选择技能(-1 退出)：";
                while(!(cin>>opt2))
                {
                    cin.clear();
                    cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                    cout<<"输入无效，请输入一个有效的整数: ";
                }
                if(opt2==-1)
                {
                    break;
                }
                if(jineng[opt2][2]==0)
                {
                    cout<<"该技能已使用！"<<endl;
                    continue;
                }
                cout<<"请输入你要使用技能的棋子位置（行 列）：";
                while(!(cin>>sx>>sy))
                {
                    cin.clear();
                    cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                    cout<<"输入无效，请输入两个有效的整数: ";
                }
                if(amap[sx][sy]<=10 || amap[sx][sy]>20)
                {
                    cout<<"你只能选择玩家 2 的棋子！"<<endl;
                    continue;
                }
                jineng[opt2][2]=0;
                if(opt2==1)
                {
                    cout<<"棋子"<<amap[sx][sy]<<"使用加 E 技能！"<<endl;
                    qizi[amap[sx][sy]]=qizi[amap[sx][sy]]+ee;
                }
                if(opt2==2)
```

```
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"使用求逆技能！"<<endl;
                        qizi[amap[sx][sy]].fuzhini();
                    }
                    if(opt2==3)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"使用转置技能！"<<endl;
                        qizi[amap[sx][sy]].zhuanzhi();
                    }
                    if(opt2==4)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"使用对角化技能！"<<endl;
                        qizi[amap[sx][sy]].fuzhiduijiao();
                    }
                    if(opt2==5)
                    {
                        cout<<"棋子"<<amap[sx][sy]<<"使用加负 E 技能！"<<endl;
                        ee=-ee;
                        qizi[amap[sx][sy]]=qizi[amap[sx][sy]]+ee;
                        ee=-ee;
                    }
                }
            }
        }

        cout<<"请输入棋子移动信息（起始行 起始列 目标行 目标列）（不可移动 18 号）：";
        while(!(cin>>sx>>sy>>ex>>ey))//玩家 1 最后移动
        {
            cin.clear();
            cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
            cout<<"输入无效，请输入四个有效的整数: ";
        }
        while(1)
        {
            if(amap[sx][sy]>=11 && amap[sx][sy]<=20 && amap[sx][sy]!=18 &&
(sx==ex || sy==ey) && sx>0 && sx<=5 && sy>0 && sy<=5 && ex>0 && ex<=5 &&
ey>0 && ey<=5)
            {
                break;
            }
            cout<<"无效的移动，请重新输入：";
            while(!(cin>>sx>>sy>>ex>>ey))
            {
```

```
                cin.clear();
                cin.ignore(std::numeric_limits<std::streamsize>::max(),'\n');
                cout<<"输入无效，请输入四个有效的整数: ";
            }
        }
        num=amap[sx][sy];
        amap[sx][sy]=-1;
        while(1)
        {
            if(sx==ex && sy==ey)
            {
                break;
            }
            sx=sx+(ex-sx)/(abs(ex-sx));
            sy=sy+(ey-sy)/(abs(ey-sy));
            if(amap[sx][sy]!=-1)
            {
                break;
            }
        }
        if(amap[sx][sy]==-1)
        {
            amap[sx][sy]=num;
        }
        else
        {
            if(amap[sx][sy]>=11)
            {
                qizi[amap[sx][sy]]=qizi[amap[sx][sy]]+qizi[num];
            }
            else
            {
                qizi[amap[sx][sy]]=qizi[amap[sx][sy]]*qizi[num];
            }
            alivecnt--;
        }
        cout<<"移动完成！"<<endl;
    }
    cout<<"===== 游戏结束 ====="<<endl;
    cout<<"最终对决：3 号棋子(玩家 1) vs 18 号棋子(玩家 2)"<<endl;
    cout<<"3 号棋子的行列式为："<<qizi[3].hanglieshi()<<endl;
    cout<<"18 号棋子的行列式为："<<qizi[18].hanglieshi()<<endl;
    if(qizi[3].hanglieshi()<qizi[18].hanglieshi())
    {
```

```
        cout<<"玩家 2 获胜！"<<endl;
    }
    if(qizi[3].hanglieshi()>qizi[18].hanglieshi())
        cout<<"玩家 1 获胜！"<<endl;
    }
    if(qizi[3].hanglieshi()==qizi[18].hanglieshi())
    {
        cout<<"平局！"<<endl;
    }
    return ;
```

可视化：

```
enum GameState {
    SELECTING,          // 选择棋子
    MOVING,             // 移动棋子
    USING_SKILL,        // 使用技能
    GAME_OVER           // 游戏结束
};

int amap[20][20], alivecnt, currentPlayer = 1;
bool jineng[10][5]={0};//1.加 E 2.求逆  3.转置  4.对角  5.减 E
Matrix ee(4);
Matrix qizi[30];
GameState gameState=SELECTING;
int selectedRow =-1,selectedCol = -1;
int skillType = 0;   // 当前选择的技能类型
bool hasOperated = false;   // 标记该回合是否已经操作

// 按钮类
class Button {
    public:
    int x, y, width, height;
    const char* text;
    bool clicked;
};

Button buttons[11];   // 存储所有按钮
void createButtons() {
    // 功能按钮
    buttons[0]={ 50, 300, 100, 30, "查看信息", false };
    buttons[1]={ 160, 300, 100, 30, "使用技能", false };
    buttons[2]={ 270, 300, 100, 30, "移动棋子", false };
    buttons[3]={ 380, 300, 100, 30, "结束回合", false };
```

```
    // 技能按钮
    buttons[4]={ 50, 340, 100, 30, "加 E", false };
    buttons[5]={ 160, 340, 100, 30, "求逆", false };
    buttons[6]={ 270, 340, 100, 30, "转置", false };
    buttons[7]={ 380, 340, 100, 30, "对角化", false };
    buttons[8]={ 490, 340, 100, 30, "减 E", false };
    buttons[9]={ 600, 300, 100, 30, "游戏结束", false };
    buttons[10]={ 710, 300, 100, 30, "查看规则", false };
}

// 初始化游戏
void initGame() {
    // 初始化技能
    for(int i = 1; i <= 5; i++) {
        jineng[i][1] = 1;
        jineng[i][2] = 1;
    }
    // 初始化地图
    for(int i=1;i<=5;i++) {
        for(int j=1;j<=5;j++) {
            amap[i][j]=-1;
        }
    }

    // 初始化玩家 1 的棋子
    for(int i=1;i<=2;i++) {
        for (int j=1;j<=5;j++) {
            amap[i][j]=(i-1)*5+j;
        }
    }

    // 初始化玩家 2 的棋子
    for(int i=4;i<=5;i++) {
        for(int j=1;j<=5;j++) {
            amap[i][j]=(i-3)*5+j+5;
        }
    }

    // 初始化单位矩阵 E
    for(int i=1;i<=4;i++) {
        for(int j=1;j<=4;j++) {
            if(i==j){
                ee.a[i][j]=1.0;
            }
```

```
                else{
                    ee.a[i][j]=0;
                }
            }
        }
        // 初始化随机数种子
        srand(time(NULL));
        for (int i=1;i<=20;i++) {
            qizi[i]=Matrix(4);
        }

        alivecnt=20;
        gameState=SELECTING;
        currentPlayer=1;
        selectedRow=-1;
        selectedCol=-1;
        hasOperated=false;
}
// 绘制按钮（使用 windows GDI）
void drawButtons(HDC hdc) {
        for (int i = 0; i < 11; i++) {
            Button &btn = buttons[i];
            // 当前为使用技能状态的话只绘制技能按钮
            if (i >= 4 && gameState != USING_SKILL&&i<=8)
                continue;
            // 绘制按钮背景
            HBRUSH brush = CreateSolidBrush(btn.clicked? RGB(150, 150, 255) : RGB(200,
200, 255)); // 点击状态为浅蓝色，未点击状态为浅灰色
            RECT rect;
            rect.left = btn.x;
            rect.top = btn.y;
            rect.right = btn.x + btn.width;
            rect.bottom = btn.y + btn.height; // 确定按钮的大小 和位置
            FillRect(hdc, &rect, brush); // brush 填充
            DeleteObject(brush); // 绘制完成后删除对象，刷新系统资源节约
            // 绘制按钮边框
            HPEN pen = CreatePen(PS_SOLID, 2, RGB(0, 0, 0)); // 创建一个黑色的 2 像
素宽的实线画笔
            HPEN oldPen = (HPEN)SelectObject(hdc, pen);
            Rectangle(hdc, btn.x, btn.y, btn.x + btn.width, btn.y + btn.height);
            SelectObject(hdc, oldPen);
            DeleteObject(pen);

            // 绘制按钮文本
```

```
            SetBkMode(hdc, TRANSPARENT);
            TextOut(hdc, btn.x + 10, btn.y + 8, btn.text, strlen(btn.text));
        }
    }
    // 绘制游戏界面
    void drawGame(HDC hdc) {
        // 绘制背景
        HBRUSH bgBrush=CreateSolidBrush(RGB(240,240,240));
        RECT rect;
        rect.left=0;
        rect.top=0;
        rect.right=1200;
        rect.bottom=1200;
        FillRect(hdc,&rect,bgBrush);
        DeleteObject(bgBrush);
        // 绘制标题
        SetBkMode(hdc,TRANSPARENT);
        HFONT
    titleFont=CreateFont(24,0,0,0,FW_BOLD,FALSE,FALSE,FALSE,DEFAULT_CHARSET,
    OUT_DEFAULT_PRECIS,                                    CLIP_DEFAULT_PRECIS,
    DEFAULT_QUALITY,DEFAULT_PITCH|FF_DONTCARE,"Arial");
        HFONT oldFont=(HFONT)SelectObject(hdc,titleFont);
        TextOut(hdc,200,10,"矩阵对战游戏",strlen("矩阵对战游戏"));
        SelectObject(hdc,oldFont);
        DeleteObject(titleFont);

        // 绘制当前玩家信息
        char playerInfo[50];
        sprintf(playerInfo,"当前玩家: %s",currentPlayer==1?"玩家 1" : "玩家 2");
        TextOut(hdc, 20, 10, playerInfo, strlen(playerInfo));

        // 显示游戏状态
        const char *stateText="";
        switch (gameState){
        case SELECTING: stateText="状态: 选择棋子";
       break;
        case MOVING:stateText="状态: 移动棋子";
       break;
        case USING_SKILL:stateText= "状态: 使用技能"; break;
        case GAME_OVER: stateText = "状态: 游戏结束"; break;
        }
        TextOut(hdc,20,40,stateText,strlen(stateText));

        // 绘制棋盘
```

```
int cellSize=50;
for (int i=1;i<=5;i++) {
    for (int j=1;j<=5;j++) {
        int x=(j-1)*cellSize+50;
        int y=(i-1)*cellSize+60;

        // 绘制棋盘格子
        HBRUSH brush;
        if (selectedRow==i&&selectedCol==j) {
            brush=CreateSolidBrush(RGB(255,255,150)); // 选中的格子为黄色
        }
        else if((i+j)%2==0){
            brush=CreateSolidBrush(RGB(220,220,220)); // 浅色
        }
        else {
            brush = CreateSolidBrush(RGB(180, 180, 180)); // 深色
        }
        RECT rect;
        rect.left=x;
        rect.top=y;
        rect.right=x+cellSize;
        rect.bottom=y+cellSize;
        FillRect(hdc,&rect,brush);
        DeleteObject(brush);

        // 绘制边框
        HPEN pen=CreatePen(PS_SOLID,1,RGB(0,0,0));
        HPEN oldPen=(HPEN)SelectObject(hdc,pen);
        Rectangle(hdc,x,y,x+cellSize,y+cellSize);
        SelectObject(hdc,oldPen);
        DeleteObject(pen);

        // 绘制棋子
        if(amap[i][j]!=-1){
            char buffer[10];
            sprintf(buffer,"%d",amap[i][j]);
            // 绘制棋子编号
            TextOut(hdc,x+cellSize/2-5,y+cellSize/2-8,buffer,strlen(buffer));
        }
    }
}

// 绘制按钮
drawButtons(hdc);
```

```
        // 显示选中棋子的信息
        if (selectedRow!=-1&&selectedCol!=-1&&amap[selectedRow][selectedCol]!=-1){
            int Id=amap[selectedRow][selectedCol];
            char info[100];
            sprintf(info,"选中棋子: %d",Id);
            TextOut(hdc, 350, 60, info, strlen(info));
            sprintf(info, "行列式: %.2f", qizi[Id].hanglieshi());
            TextOut(hdc, 350, 80, info, strlen(info));
            sprintf(info, "秩: %d", qizi[Id].zhi());
            TextOut(hdc, 350, 100, info, strlen(info));
        }
}
// 显示矩阵信息对话框
void showMatrixInfo(HWND hwnd,int row,int col){
        if (row==-1||col==-1||amap[row][col]==-1)
       return;
        int pieceId=amap[row][col];
        Matrix& matrix = qizi[pieceId];
        char title[50];
        sprintf(title,"棋子 %d 的矩阵信息",pieceId);
        char info[1000] = "";
        char buffer[200];
        sprintf(buffer,"矩阵:\n");
        strcat(info,buffer);
        for (int i=1;i<=4;i++){
            for(int j=1;j<=4;j++){
                sprintf(buffer,"%8.2f",matrix.a[i][j]);
                strcat(info,buffer);
            }
            strcat(info,"\n");
        }
        sprintf(buffer,"\n 行列式: %.2f\n",matrix.hanglieshi());
        strcat(info,buffer);
        sprintf(buffer,"秩: %d\n",matrix.zhi());
        strcat(info, buffer);
        MessageBox(hwnd,info,title,MB_OK);
}
// 移动棋子
// 移动棋子
void movePiece(int fromRow,int fromCol,int toRow,int toCol,HWND hwnd) {
        if(fromRow==-1||fromCol==-1||amap[fromRow][fromCol]==-1)
            return;
        if(hasOperated){
       MessageBox(hwnd,"本回合已进行操作，无法再次移动","提示", MB_OK |
```

```
MB_ICONINFORMATION);
            return;
        }
        int pieceId =amap[fromRow][fromCol];
        // 判断是否为当前玩家的棋子
        bool isCurrentPlayerPiece =false;
        if (currentPlayer==1&&pieceId>=1&&pieceId<=10){
            isCurrentPlayerPiece=true;
        }else if(currentPlayer==2&&pieceId>=11&&pieceId<=20){
            isCurrentPlayerPiece=true;
        }
        if(pieceId==3||pieceId==18){
            MessageBox(hwnd," 不 能 移 动 棋 子   3  和   18!"," 提 示 ",MB_OK |
MB_ICONINFORMATION);
            gameState=SELECTING;
            return;
        }
        if(!isCurrentPlayerPiece){
            MessageBox(hwnd," 只 能 移 动 自 己 的 棋 子 !"," 提 示 ",MB_OK |
MB_ICONINFORMATION);
            gameState=SELECTING;
            return;
        }
        // 判断移动是否合法，只能垂直或水平移动
        if ((fromRow!=toRow&&fromCol!=toCol)||fromRow==toRow&&fromCol==toCol){
            MessageBox(hwnd, " 只 能 垂 直 或 水 平 移 动 棋 子 !"," 提 示 ",MB_OK |
MB_ICONINFORMATION);
            return;
        }

        // 判断目标位置是否有棋子
        if(amap[toRow][toCol]!=-1){
            int targetId=amap[toRow][toCol];
            bool isSamePlayer=false;
            // 判断目标棋子是否为同一玩家
            if(currentPlayer==1&&targetId>=1&&targetId<=10){
                isSamePlayer=true;
            } else if(currentPlayer==2&&targetId>=11&&targetId<=20){
                isSamePlayer=true;
            }
            // 同一玩家棋子合并
            if (isSamePlayer){
                qizi[targetId]=qizi[targetId]+qizi[pieceId];
                amap[fromRow][fromCol]=-1; // 原位置清空
```

```
                alivecnt--;
                MessageBox(hwnd," 棋 子 合 并 成 功 !"," 提 示 ", MB_OK |
MB_ICONINFORMATION);
            }
            // 不同玩家棋子对战
            else {
                // 对战逻辑，比较行列式绝对值大小，大的获胜
                double attackerDet = qizi[pieceId].hanglieshi();
                double defenderDet = qizi[targetId].hanglieshi();
                if (fabs(attackerDet)>fabs(defenderDet)) {
                    // 攻击方获胜
                    amap[toRow][toCol] = pieceId;
                    amap[fromRow][fromCol] = -1;
                    alivecnt--;
                    MessageBox(hwnd, "攻击成功，对方棋子被移除!", "提示", MB_OK
| MB_ICONINFORMATION);
                } else if (fabs(attackerDet) < fabs(defenderDet)) {
                    // 防守方获胜
                    alivecnt--;
                    MessageBox(hwnd, "攻击失败，自己的棋子被移除!", "提示",
MB_OK | MB_ICONINFORMATION);
                } else {
                    // 平局，双方棋子都移除
                    amap[fromRow][fromCol] = -1;
                    amap[toRow][toCol] = -1;
                    alivecnt -= 2;
                    MessageBox(hwnd, "双方平局，双方棋子都被移除!", "提示",
MB_OK | MB_ICONINFORMATION);
                }
            }
        } else {
            // 目标位置没有棋子，直接移动
            amap[toRow][toCol] = pieceId;
            amap[fromRow][fromCol] = -1;
        }

        // 检查是否有一方剩余棋子小于 3 个
        int player1Count = 0, player2Count = 0;
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= 5; j++) {
                if (amap[i][j] >= 1 && amap[i][j] <= 10) {
                    player1Count++;
                } else if (amap[i][j] >= 11 && amap[i][j] <= 20) {
                    player2Count++;
```

```
                }
            }
        }

        if (player1Count < 3 || player2Count < 3) {
            double det3 = qizi[3].hanglieshi();
            double det18 = qizi[18].hanglieshi();
            if (det3 > det18) {
                MessageBox(hwnd, "玩家 1 获胜!（棋子 3 行列式大于棋子 18）", "游戏结束", MB_OK | MB_ICONINFORMATION);
                gameState = GAME_OVER;
            } else if (det3 < det18) {
                MessageBox(hwnd, "玩家 2 获胜!（棋子 18 行列式大于棋子 3）", "游戏结束", MB_OK | MB_ICONINFORMATION);
                gameState = GAME_OVER;
            } else {
                MessageBox(hwnd, "平局!（棋子 3 和棋子 18 行列式相等）", "游戏结束", MB_OK | MB_ICONINFORMATION);
                gameState = GAME_OVER;
            }
        }

        hasOperated = true;

        // 切换玩家
        currentPlayer = 3 - currentPlayer;
        gameState = SELECTING;
        selectedRow = -1;
        selectedCol = -1;
        hasOperated = false; // 重置操作标记
}
void useSkill(int skillType, int row, int col, HWND hwnd) {
        if (row == -1 || col == -1 || amap[row][col] == -1) return;

        if (hasOperated) {
            MessageBox(hwnd, "该回合已进行操作，无法再次移动", "提示", MB_OK | MB_ICONINFORMATION);
            return;
        }

        int pieceId = amap[row][col];

        // 检查技能是否可用
        if (!jineng[skillType][currentPlayer]) {
```

```
            MessageBox(hwnd, "该技能不可使用!", "提示", MB_OK |
MB_ICONINFORMATION);
            return;
        }

        // 判断是否为当前玩家的棋子
        bool isCurrentPlayerPiece = false;
        if (currentPlayer == 1 && pieceId >= 1 && pieceId <= 10) {
            isCurrentPlayerPiece = true;
        } else if (currentPlayer == 2 && pieceId >= 11 && pieceId <= 20) {
            isCurrentPlayerPiece = true;
        }

        if (!isCurrentPlayerPiece) {
            MessageBox(hwnd, "只能对自己的棋子使用技能!", "提示", MB_OK |
MB_ICONINFORMATION);
            return;
        }

        // 使用技能
        switch (skillType) {
        case 1:
            qizi[pieceId] = qizi[pieceId] + ee;
            MessageBox(hwnd, "加 E 技能使用成功!", "提示", MB_OK |
MB_ICONINFORMATION);
            break;
        case 2:
            qizi[pieceId].fuzhini();
            MessageBox(hwnd, "求逆技能使用成功!", "提示", MB_OK |
MB_ICONINFORMATION);
            break;
        case 3:
            qizi[pieceId].zhuanzhi();
            MessageBox(hwnd, "转置技能使用成功!", "提示", MB_OK |
MB_ICONINFORMATION);
            break;
        case 4:
            qizi[pieceId].fuzhiduijiao();
            MessageBox(hwnd, "对角化技能使用成功!", "提示", MB_OK |
MB_ICONINFORMATION);
            break;
        case 5:
            qizi[pieceId] = qizi[pieceId]   -ee;
            MessageBox(hwnd, "减 E 技能使用成功!", "提示", MB_OK |
```

```
MB_ICONINFORMATION);
            break;
        }

        // 标记技能已使用
        jineng[skillType][currentPlayer] = 0;
        hasOperated = true; // 标记已操作

        // 切换玩家
        currentPlayer = currentPlayer == 1 ? 2 : 1;
        gameState = SELECTING;
        selectedRow = -1;
        selectedCol = -1;
        hasOperated = false; // 重置操作标记

        InvalidateRect(hwnd, NULL, TRUE);
}
// 窗口过程函数
LRESULT CALLBACK RulesWindowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam) {
        switch (msg) {
        case WM_PAINT: {
            PAINTSTRUCT ps;
            HDC hdc = BeginPaint(hwnd, &ps);

            char buffer[1024] = {};
            FILE* file = fopen("rules.txt", "r");
            if (file) {
                fread(buffer, sizeof(char), sizeof(buffer) - 1, file);
                fclose(file);
            } else {
                strcpy(buffer, "游戏规则文件未找到！");
            }

            RECT rect;
            GetClientRect(hwnd, &rect);
            DrawTextA(hdc, buffer, -1, &rect, DT_LEFT | DT_WORDBREAK);

            EndPaint(hwnd, &ps);
            break;
        }
        case WM_DESTROY:
            PostQuitMessage(0);
            break;
```

```
    default:
        return DefWindowProc(hwnd, msg, wParam, lParam);
    }
    return 0;
}
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    static HINSTANCE g_hInstance = NULL;
    switch (msg) {
    case WM_CREATE:
        g_hInstance = ((LPCREATESTRUCT)lParam)->hInstance; // 保存实例句柄
        initGame();
        createButtons();
        break;
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hwnd, &ps);
        drawGame(hdc);
        EndPaint(hwnd, &ps);
        break;
    }
    case WM_LBUTTONDOWN: {
        int x = LOWORD(lParam);
        int y = HIWORD(lParam);
        // 处理按钮点击事件
        for (int i = 0; i < 11; i++) {
            Button& btn = buttons[i];
            if (x >= btn.x && x <= btn.x + btn.width && y >= btn.y && y <= btn.y + btn.height) {
                btn.clicked = true;
                if (i == 0) {
                    showMatrixInfo(hwnd, selectedRow, selectedCol);
                } else if (i == 1) {
                    gameState = USING_SKILL;
                } else if (i == 2) {
                    gameState = MOVING;
                } else if (i == 3) {
                    currentPlayer = 3 - currentPlayer;
                    gameState = SELECTING;
                    selectedRow = -1;
                    selectedCol = -1;
                    hasOperated = false;
                } else if (i >= 4 && i <= 8) {
                    skillType = i - 3;
```

```
                if (selectedRow != -1 && selectedCol != -1) {
                    useSkill(skillType, selectedRow, selectedCol, hwnd);
                } else {
                    MessageBox(hwnd, "请先选择一个棋子!", "提示", MB_OK |
MB_ICONINFORMATION);
                }
            } else if (i == 9) {
                // 处理游戏结束按钮事件
                gameState = GAME_OVER;
                char result[100];
                int player1Count = 0, player2Count = 0;
                for(int i = 1; i <= 5; i++) {
                    for(int j = 1; j <= 5; j++) {
                        if (amap[i][j] >= 1 && amap[i][j] <= 10) player1Count++;
                        else if (amap[i][j] >= 11 && amap[i][j] <= 20)
player2Count++;
                    }
                }
                if(player1Count > player2Count) {
                    sprintf(result, "游戏结束，玩家 1 获胜\n 玩家 1 剩余棋子: %d\n
玩家 2 剩余棋子: %d", player1Count, player2Count);
                } else if(player1Count < player2Count) {
                    sprintf(result, "游戏结束，玩家 2 获胜\n 玩家 1 剩余棋子: %d\n
玩家 2 剩余棋子: %d", player1Count, player2Count);
                } else {
                    sprintf(result, "游戏结束，平局\n 玩家 1 剩余棋子: %d\n 玩家 2
剩余棋子: %d", player1Count, player2Count);
                }
                MessageBox(hwnd, result, "游戏结束", MB_OK |
MB_ICONINFORMATION);
            } else if (i == 10) { // 规则按钮
                    const char RULES_CLASS_NAME[] = "RulesWindowClass";
                    WNDCLASS rulesWC = {};
                    rulesWC.lpfnWndProc = RulesWindowProc;
                    rulesWC.hInstance = g_hInstance;
                    rulesWC.lpszClassName = RULES_CLASS_NAME;
                    rulesWC.hCursor = LoadCursor(NULL, IDC_ARROW);
                    rulesWC.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

                    ATOM rulesClassAtom = RegisterClass(&rulesWC);
                    if (!rulesClassAtom) {
                        MessageBox(NULL, "注册规则窗口类失败!", "错误",
MB_OK | MB_ICONERROR);
                        break;
```

```
                    }

                    HWND rulesHwnd = CreateWindowEx(
                        0,
                        RULES_CLASS_NAME,
                        "游戏规则",
                        WS_OVERLAPPEDWINDOW,
                        CW_USEDEFAULT, CW_USEDEFAULT, 400, 300,
                        NULL,
                        NULL,
                        g_hInstance,
                        NULL
                    );

                    if (rulesHwnd) {
                        ShowWindow(rulesHwnd, SW_SHOW);
                        UpdateWindow(rulesHwnd);
                    } else {
                        MessageBox(NULL, "创建规则窗口失败!", "错误",
MB_OK | MB_ICONERROR);
                    }
                }
            }
        }
    // 处理棋子点击事件
    int cellSize = 50;
    int boardX = 50;
    int boardY = 60;
    if (x >= boardX && x < boardX + 5 * cellSize &&
        y >= boardY && y < boardY + 5 * cellSize) {
        int col = (x - boardX) / cellSize + 1;
        int row = (y - boardY) / cellSize + 1;
        if (gameState == SELECTING) {
            selectedRow = row;
            selectedCol = col;
        } else if (gameState == MOVING) {
            if (selectedRow != -1 && selectedCol != -1) {
                movePiece(selectedRow, selectedCol, row, col, hwnd);
                selectedRow = -1;
                selectedCol = -1;
            }
        }
        InvalidateRect(hwnd, NULL, TRUE);
    }
```

```
        break;
}
        case WM_DESTROY:
            PostQuitMessage(0);
            break;
        default:
            return DefWindowProc(hwnd, msg, wParam, lParam);
        }
        return 0;
}
```

}1.5 测试与运行

Test.out

test1.in test2.in

1.6 调试记录

版本 1.0
1.加法运算符重载返回值错误
时间：2025.4.26 21:48
内容：重载的加法运算符 operator+ 没有返回正确的矩阵对象。operator+ 函数没有返回一个新的矩阵对象，而是直接修改了当前对象的值，这会导致加法操作不符合预期。
解决方案：创建一个新的矩阵对象，并将相加的结果存储在该对象中，然后返回该对象。

2. 求逆函数除零错误隐患
时间：2025.4.27 20:22
内容：在计算矩阵的逆时，qiuni() 函数中的部分循环逻辑可能导致除零错误。在消元过程中，可能会出现主元为零的情况，导致除零错误。
解决方案：在消元过程中，检查主元是否为零，如果为零则尝试交换行，确保主元不为零。

版本 1.21
1. 在 Matrix 类的构造函数里使用
时间：2025.5.9 10:05
问题内容： double(rand())/RAND_MAX*10.0 随机生成矩阵元素时，每次运行程序生成的矩阵元素值都是相同的序列。这是因为 rand() 函数在未设置不同种子的情况下，每次启动程序都会依据相同的初始值来生成随机数序列。
解决方案：在 work 函数里添加 srand(time(NULL)); 代码，借助当前时间作为随机数生成器的种子，从而保证每次运行程序时都能生成不同的随机数序列。

2. 生成的矩阵元素值范围不符合预期
时间：2025.5.9 10:20
问题内容：原本期望元素值在 0 到 10 之间，然而实际运行时发现部分元素值超出了这个范围。经检查，是由于 double(rand())/RAND_MAX*10.0 计算时，rand() 函数返回的是一个整数，在进行除法运算时可能会产生精度丢失的问题。
解决方案：将 double(rand())/RAND_MAX*10.0 改为 (double)rand() / RAND_MAX * 10.0，明确进行类型转换，确保计算结果的精度，从而保证生成的矩阵元素值在 0 到 10 之间。

版本 2.1
1. 主要问题：使用技能后未触发玩家切换，技能使用后仍处于当前玩家回合。
时间：2025 . 5. 13 21:22
解决方案：在 useSkill 函数末尾添加切换玩家代码，包括切换玩家编号（currentPlayer = 3 - currentPlayer;）、重置游戏状态（gameState = SELECTING;）和操作标记（hasOperated = false;）。

2.主要问题：游戏结束后仍可点击按钮进行操作，出现逻辑混乱。
时间：2025 . 5. 13 21:48
解决方案：在 WndProc 的按钮点击事件中增加 gameState == GAME_OVER 判断，若游戏已结束（GAME_OVER 状态），则弹出提示框并禁止执行操作。

3.主要问题：修改获胜条件后，未正确禁止棋子 3 和 18 的移动。
时间：2025 . 5. 13 21:59
解决方案：在 movePiece 函数开头添加条件判断，若选中棋子为 3 或 18，弹出提示框（如“禁止移动棋子 3 和 18！”）并终止移动流程（return;）。

4.主要问题：玩家回合切换时，界面未及时更新当前玩家信息（如界面仍显示上一玩家）。
时间：2025 . 5. 13 23:15
解决方案：在切换玩家后调用 InvalidateRect(hwnd, NULL, TRUE);强制刷新界面，确保当前玩家信息实时更新。

版本 2.2
1. 主要问题：若玩家未水平或竖直移动棋子，提示结束后无法进行继续选择操作
   时间：2025.5.18 21:29
   解决方案： 在 movePiece 函数，判断是否水平或竖直移动棋子函数部分末尾，加上游戏状态更新 gameState=SELECTING;
2. 主要问题：若玩家移动棋子至行列式比己方棋子行列式大的位置，无法自动将己方棋子抹去；
   时间：2025.5.18 21：43
   解决方案：在 movePiece 函数相应部位，添加 alivecnt--;amap[fromRow][fromCol] = -1;即存活棋子数减一，原位置棋子消除。

3. 主要问题：点击“游戏结束”按钮后，无法自动结束游戏。
   时间:2025.5.18 22:07
   解决方案：在 LRESULT CALLBACK WndProc 函数 WM_LBUTTONDOWN（按钮点击部分）游戏结束按钮选项后面加入语句：PostQuitMessage(0);

1.7 版本记录（若有）

0.0 0.1 版本为半成品

1.0 版本可初步运行

1.1 增加功能
1.2 增加中文提示
1.21 版本加入随机生成矩阵并优化规则
2.1 版本加入可视化，增加现实可玩性
2.2 版本修复上一版本遗留 bug

2 分工情况

薄闻乐：代码主体部分撰写（40%）；
魏舟：撰写可视化代码（借助 AI），撰写报告（30%）；
尹杨阳：检查并修正代码，做 PPT（30%）；

3 心得体会

课程设计过程中的心得和体会。

薄闻乐：

从一开始时还没有清晰的想好要做什么，之后在通过作业熟悉了类的设计与使用，在程序实践中逐渐熟练了对类的编写，遇到比较困难的函数能通过一步步拆分到不同的功能函数完成。通过程序组织和数学逻辑，一个完整的矩阵类被编织出来，在使用中更为得心应手。在主程序的实现上我自己写的版本在功能调用上还有所欠缺，对类的使用不那么完全。但在团队合作中，程序一代代成长，直到最后实现运行，这段经历实在难忘。

魏舟：

这次小组作业让我收获特别多。

在技术学习方面，我遇到了不少挑战。最开始用 Windows API 编程的时候，完全摸不着头脑，什么窗口类注册、消息循环，感觉特别复杂。但经过不断查资料、看教程，我慢慢理解了，原来操作系统和程序是通过消息来交互的，这真的很神奇。还有图形绘制，我学会用 GDI 画矩形、写文本，不过一开始图形的位置和大小总是不对，经过反复尝试才调好。另外，类和对象的使用让代码更有条理，我能把按钮类封装起来，用的时候很方便。

团队合作也特别重要。我们根据大家的特长分工，有人负责算法，有人做可视化，

有人搞测试和文档。我们定期交流，一起解决问题。有一次可视化部分要调用矩阵类的方法，我们和负责算法的薄闻乐同学沟通了好久，才保证接口没问题。
解决问题的过程也让我成长了很多。图形绘制精度不够，界面看着不美观，我查了好多资料，才知道可以调整坐标来解决。消息处理逻辑出错，鼠标点击按钮没反应，我仔细分析代码、调试，最后发现是判断点击位置的逻辑有问题，改过来就好了。

这次作业让我学到了新的编程知识，也让我知道团队合作有多重要。以后我会继续努力，提高自己的能力，把程序变得更好。

尹杨洋：

参与这次的 c++项目让我获益颇丰。对战小游戏中对矩阵的处理和成员函数的设计等加强了我对面向对象编程的了解和运用水平，例如在求逆函数中要调用求秩和求对角化函数的结果，这种模块化的编程技巧给了我在很多方面新的启发；在对角化函数里使用了分层循环查找，这类方法我曾经使用时经常犯错，而现在我对它的了解更进一步了。可视化方面的程序也让我学到了课程教授内容之外的新知识。绘制图形界面，使用数组处理按钮，窗口过程函数等等，在曾经的我看来都是遥不可及的，而在为项目进行学习之后我发现其实也并没有我想象中那样困难。作为 ppt 和演示视频的制作者，制作过程需要我总览全局，将眼光放到整个程序上，这迫使着我不仅要了解程序的编程细节，也更要从总体上去体会程序中各个部分的组成与联系。这让我从与以往完全不同的全新角度认识了代码和编程，看到程序的各个模块环环相扣，我更加理解了模块化编程这一概念，感受到了编程的精妙之处。这样的合作体验，比起以往任务式地编写小程序更能让我收获良多。